UNIT TESTING

Amy Krause and Mike Jackson
EPCC

What is unit testing?

- Testing the smallest possible units of our software
  - Functions
  - Subroutines
  - Methods
  - Classes
- Compared to regression tests
  - Quicker to run – don't have to build all our software to test one function
  - Easier to guarantee that all functions are tested

Advantages

- Encourages good design
  - High cohesion
  - Loose coupling
  - Code has to be tested in small chunks
  - Makes you consider how you want a function to behave
  - If code is hard to write tests for this can highlight a need for redesign
- A form of documentation
  - Live code examples
  - Documentation which you can automatically check is correct

Code-then-test

- Develop the unit then the test
  - Write unit tests as soon as you have a unit to test
- Pros
  - Fits closely with the traditional way of developing software
  - Intuitive
- Cons
  - Focus on what code does, not what it should do
  - Testing is often viewed as a nice-if (if time permits…)
- Test early, test often

Test-then-code

- Test-driven development
  - Think about what a unit needs to do
  - Write tests to test its functionality, which fail, as there is no code
  - Write code to make the tests pass
  - Clean up, refactor, the code
  - Red – Green - Refactor
- Pros
  - Focus on what code should do, not what it does
  - Testing becomes an integral part of development
  - All code has tests
- Cons
  - Counter-intuitive
  - Requires discipline

epcc

xUnit test frameworks

- CUnit, CppUnit, FRUIT, JUnit, py.test, …
- Automate the process of writing and running unit tests
- Assertions compare actual results to expected results
- Organise test function, method, class, module directories
  - `src/maths/Sin.java`
  - `test/maths/SinTest.java`
- Automatically find and run unit tests
- Advantages
  - Run one or more unit tests with a single command
  - Fast, so they can be run frequently
  - Repeatable, so developers (should) get the same results
- Automated build + test = fail fast

epcc

Tests are code

- Review tests to detect tests that
  - Pass when they should fail, false positives
  - Fail when they should pass, false negatives
  - Don't test anything

```
def test_critical_correctness():
    ...lots of set up and configuration...
    # TODO - will add assertions later!
    pass
```